American University of Sharjah
College of Engineering
Computer Science & Engineering Dept.
P.O. Box 26666
Sharjah, UAE

Instructor: Dr. Michel Pasquier
Office: ESB-2057
Phone: 971-6-515-2883
E-mail: mpasquier@aus.edu
Semester: xxxx xxxx

CMP 321 PROGRAMMING LANGUAGES

MIDTERM EXAM PAPER #2 – SAMPLE

Date: xxxx xx xxxxxxx xxxx (1 hour)

Student ID: _____ **Student Name:** _____

Instructions: Write both your *name and ID* above. Read the questions carefully; write your answers clearly in the space provided. You can use draft paper, if necessary, but it must be returned with this paper. This is a *close book exam:* neither books or notes are allowed, nor is any communication with anyone but the examiner.

For Instructor's Use Only:

Q1	Python classes and functions	/ 8
Q2	Python higher-order functions	/ 12
Q3	Python iterators and generators	/ 12
Q4	Language grammars, parsing	/ 10
Q5	Attribute grammar, semantics	/ 8
Q6	Regular expressions	

Total: / **50**

Question 1: Write a minimal Python class for `Square` shapes so that the code below works exactly as shown and the following specs are met: The width attribute is *always* an integer value; it is *hidden* from users. The *default* width is initially ten but we can *change* it anytime. Make sure to write proper Pythonic code. (8 marks)

```
s1 = Square()                  # creates a new square of default width
s2 = Square(5)                 # creates a new square with custom width
print(len(s2))                 # prints the square's current width
s3 = Square('ten')             # exception caught, error message printed
> error: width must a number
```

Question 2: **(a)** For each task below, write a single statement using Python functions and higher-order functions (but *not* comprehensions) to yield the result. (5 marks)

– Calculate the average length of all lists contained in a tuple `t`

e.g., `t = ([1],[2,3],[4,5,6]) -> 2`

– Add all the numbers in a string which are separated with a `+` sign

e.g., `s = "1.7 + 5 + 3.4 + 2" -> 12.1`

(b) Use a Python *comprehension* to realize each of the tasks below. (5 marks)

– Create a list of all the divisors of a given number `n` (excl. `1` and `n`)

e.g., `n = 99 -> [3, 9, 11, 33]`

– Create a list of the squares of all numbers in a list, ignoring non-numbers

e.g. `l = [2, 'str', {}, 35/5, int('5'), (3**2,), 11]`
`-> [4, 49.0, 25, 121]`

(c) Explain exactly how comprehensions are related to higher-order functions in Python, using an example of your choosing. (2 marks)

Question 3: (a) Examine the function below and give the output of the two function calls. Explain briefly how this function works, justifying the use of `iter` and `try`/ `except` in the code. Which well-known function does it implement? (6 marks)

```
def fun (iterable, op=operator.add):
    it = iter(iterable)
    try: result = next(it)
    except StopIteration: return
    yield result
    for element in it:
        result = op(result, element)
        yield result

for n in fun([2,9,5,4]): print(n, end=' ')

list(fun([2,9,5,4], operator.mul))
```

(b) Indicate what the following Python code does *exactly,* and why it is *inefficient.* Write a *generator function* to replace slicing and solve the problem. (6 marks)

```
for item in data[::-1] : print(item)
```

Question 4: **(a)** Explain briefly but clearly, using an example of your choosing, why the grammar of a programming language *cannot* be ambiguous. (3 marks)

(b) Draw two distinct *parse trees* to show that the `sentence 'small cats and dogs'` is *ambiguous* given the the BNF grammar below. (4 marks)

```
<s> -> <np>                  |   <n> -> cats | dogs | birds |
<np> -> <a> <np>             |          hamsters | turtles
<np> -> <np> and <np>        |   <a> -> large | small | red |
<np> -> <n>                  |          blue | brown | cute
```

(c) Prove that the sentence `'small red and blue birds'` is *not* in the language defined by the BNF grammar in part (b) above. (3 marks)

Question 5: The grammar below (same as in Q4) can generate sentences with *more than three adjectives*, such as `'cute small red blue birds'`, which we want to *disallow.* Explain how it can be done (i) in BNF, (ii) in EBNF, then (iii) using an Attribute Grammar. Write all necessary rules and details for the latter. (8 marks)

```
<s> -> <np>
<np> -> <a> <np>
<np> -> <np> and <np>
<np> -> <n>
```

```
<n> -> cats | dogs | birds |
       hamsters | turtles
<a> -> large | small | red |
       blue | brown | cute
```

END OF PAPER